

第086课实验手册：把NumPy迁到真实PyTorch

任务与原则

本实验不接受“PyTorch代码已经写好但没有运行”作为完成。你要实际导入CPU PyTorch，先完成三层数值对齐，再执行包含训练、验证、检查点选择与一次测试的完整流程。NumPy实现随本讲独立保存，085不是运行时依赖。

依赖分为核心计算与可选构建。核心为NumPy和CPU PyTorch；图形需要Matplotlib；Notebook和PDF的依赖另列。先按README安装环境，再在本目录执行。所有实验读取本地固定JSON，不下载数据、不调用外部服务。

```
python experiment.py --out outputs/result.json
python test_experiment.py --out outputs/test-result.json
python -O test_experiment.py --out outputs/test-result-optimized.json
python plots.py --directory outputs/figures --result outputs/result.json
```

阶段A：Python接口桥梁

先创建一个Counter类，在__init__保存value，用add方法增加它。创建两个实例，只修改其中一个，观察另一个是否改变。然后创建继承Counter的子类，调用super().init，增加自己的属性。解释self指向哪个实例，类与实例有什么区别。

用一个包含yield的函数产生两批数据。调用生成器函数后先打印对象，再用next取两次，第三次捕获StopIteration。最后比较“每轮重新调用生成器函数”与“重复使用同一个耗尽的迭代器”。Notebook内已有可运行桥梁，你也应能脱离模板重写。

阶段B：检查数据与模型契约

读取ArrayDataset的一条样本，打印特征与标签的shape、dtype和device。再从训练DataLoader取一批，观察单样本如何叠成批量。完整遍历并记录各批样本数，确认323条没有遗漏，尾批为3条。验证157条尾批29条。

创建TwoLayer(3)，列出named_parameters，每一项记录形状和元素数，总数应17。根据NumPy约定手写参数映射：W1和W2转置，偏置不转置。不要只用同一个随机种子在两个库各初始化一次，它们随机算法和默认初始化不同，不能保证参数相同。

阶段C：三层迁移审计

运行parity_audit，记录NumPy与PyTorch的logits、损失、四块梯度以及一次SGD后参数的最大绝对误差。共同条件为同样的输入与标签、明确复制的参数、CPU float64、tanh、平均交叉熵、无动量SGD学习率0.07。

验收阈值均为 $1e-12$ 。若前向已不符，先查布局与激活，不要直接查autograd；若前向符合但梯度不符，查损失归约与标签；若梯度符合但更新不符，查学习率、优化器附加参数以及是否更新了同一模型。按照最早失败阶段诊断。

阶段D：亲眼看见梯度累积与断链

对 $w=2$ 、 $L=(3w-1)^2$ 先手算梯度，再运行accumulation_audit，比较30、60、30。每次都重新前向，参数保持不变；不要在同一个已释放的图上连续backward而误把计算图复用错误当作累积规则。

在独立练习中对输出detach后尝试反向，解释报错。再演示loss.item只适合日志，不能替代原损失张量。最后检查zero_grad(set_to_none=True)后grad是None而不是装着零的Tensor，并说明两者为何需要区分。

阶段E：完整训练、验证与检查点

阅读train_epoch、evaluate和run，逐行标注哪些语句修改参数、哪些只读取参数。训练每轮先train，验证用eval与no_grad。每批清梯度、前向、交叉熵、反向、更新。在线训练损失按样本数加权；固定模型的训练损失与在线值分开保存。

180个epoch预算事先固定，验证损失选择最佳state_dict的深拷贝；训练结束恢复该状态，测试集只评价一次。记录选中的epoch和三份数据的最终指标。不要用最终测试准确率决定是否再试另一个随机种子。

阶段F：状态与副作用核验

运行state_audit，观察Dropout训练态的零比例与评价态的原样输出。再确认eval状态下仍然可能requires_grad=True，no_grad才控制记录。主网络没有Dropout或BatchNorm，因此状态演示和迁移实验分开，以免随机性污染基础对齐。

运行普通和-O版本各11项测试。确认证验不改变参数或产生新梯度、尾批统计与NumPy整体计算一致、真实PyTorch版本写入结果。框架未安装必须失败，不能以跳过测试的方式声称通过。

练习

1. 写Counter和继承它的NamedCounter。分别创建两个实例，解释实例属性、self与super的作用。
2. 写每次yield一批的生成器，说明生成器耗尽后为什么不能直接再当新epoch使用。
3. NumPy W_1 为 2×3 、 W_2 为 3×2 。写出PyTorch两个Linear的weight形状，以及参数和梯度的双向映射。
4. 手算 $w=2$ 时 $L=(3w-1)^2$ 的梯度。两次不清梯度的backward得到多少？如果中间step改变 w ，还一定翻倍吗？
5. 两批验证数据大小32与3，批均损失0.2与1.0。求正确平均损失，并解释0.6为何错误。
6. 解释model.eval与torch.no_grad的区别。指出验证漏掉其中一个会在什么情形下出问题。
7. 为什么只比较NumPy与PyTorch最终准确率不能完成迁移验收？设计最少三层检查，并列出必须对齐的条件。

8. 35条逻辑批量拆成32与3，在不改参数的情况下累积梯度，如何让它等于整个35条样本的平均损失梯度？说明需要清梯度和step的时机。
9. 为什么保存最佳检查点需要深拷贝？验证选择之后，测试还应承担什么职责？

提交结果

提交Python桥梁短程序、数据与参数形状记录、三层误差表、梯度累积解释、完整训练JSON、两份测试日志和四幅图。结论要区分“迁移一致”“训练成功”“外推可靠”三个命题，不能从前两项跳到第三项。